



Universidad Pública de Navarra
Nafarroako Unibertsitate Publikoa

DESPLIEGUE DE SERVICIOS MULTIMEDIA
Máster Universitario en Ingeniería de Telecomunicación

AppGaztaroa
Commit 07: “Botones o Iconos”

Marko Galarza Galarza
Mikel Sagues García

Botones o Iconos

Commit 07: “Botones o Iconos”

En este capítulo vamos a introducir en nuestra aplicación uno de los mecanismos más habituales de interactuar con el usuario: los botones. Sin embargo, vamos a hacerlo sin hacer uso del componente *Button* de React Native, ni de ninguna otra librería, ya que nos valdremos de Iconos para implementar dicha funcionalidad.

Además, completaremos la vista de detalle de las excursiones, renderizando una nueva *Card* en la que se mostrarán los comentarios de los participantes de la excursión.

1. Añadir comentarios a la vista del componente *DetalleExcursion*

En primer lugar, crearemos un nuevo *Card* para renderizar los comentarios asociados al detalle de cada una de las excursiones. Para ello:

- Crearemos una nueva clase funcional de nombre *renderComentario(props)* dentro del fichero *DetalleExcursionComponent.js*, de manera análoga a lo que hicimos con la clase funcional *renderExcursion(props)*.
- La información a mostrar será obtenida del fichero JavaScript *comentarios.js* que se adjunta a este documento y que copiaremos en la carpeta *comun* dentro de nuestra aplicación. En el renderizado de cada comentario deberán mostrarse el texto del comentario, la valoración en estrellas, el autor y la fecha/hora del comentario. Esta última no debe mostrarse en bruto, sino correctamente formateada para que resulte legible al usuario.
- A continuación se muestran algunas de las modificaciones que debemos hacer sobre el código fuente del fichero *DetalleExcursionComponent.js*:

```
[...]
import { Text, View, ScrollView, FlatList } from 'react-native';
import { COMENTARIOS } from '../comun/comentarios';

[...]

function RenderComentario(props) {

  const comentarios = props.comentarios;

  return (
    <Card style={styles.card}>
      <Card.Title>[...]</Card.Title>
      <Card.Content/>
      [... renderizar aquí el contenido de los comentarios ...]
    </Card>
  );
}
```

```
    );  
  }  
  
  [...]  
  
    this.state = {  
    [... actualizar estado para tener acceso a los comentarios ...]  
    };  
  
  [...]  
  
  render(){  
    const {excursionId} = this.props.route.params;  
    return(  
      <ScrollView>  
        <RenderExcursion  
          excursion={this.state.excursiones[+excursionId]}  
        />  
        <RenderComentario  
          comentarios={this.state.comentarios.filter((comentario) =>  
> comentario.excursionId === excursionId)}  
        />  
      </ScrollView>  
    );  
  }  
  
  [...]
```

2. Botones o Iconos

A continuación, añadiremos nuestros primeros botones a la aplicación, empleando el componente *IconButton* de la librería *React Native Paper*.

Nuestra “excusa” para crear este primer botón será el permitir al usuario que marque como “favorita” una de las excursiones del Club Deportivo Gaztaroa. Este hecho, además de modificar la apariencia del botón para mostrar que ya ha sido clicado (y que, en consecuencia, la excursión en la que estamos la consideramos como “favorita”), también debe implementar una sencilla lógica que permita a la aplicación saber que el botón ya ha sido clicado.

Para conseguir esta funcionalidad, el código del fichero *DetalleExcursionComponent.js* debe actualizarse siguiendo la siguiente guía, a partir de lo que ya hemos construido en el primer apartado de este capítulo:

```
[...]  
import { Card, Text, Divider, IconButton } from 'react-native-paper';  
[...]
```

```
<View style={styles.iconoContainer}>
  <IconButton
    icon={props.favorita ? 'heart' : 'heart-outline'}
    size={28}
    onPress={() =>
      props.favorita
        ? console.log('La excursión ya se encuentra entre las
favoritas')
        : props.onPress()
    }
  />
</View>
/>

[...]
```

```
  this.state = {
    [...]
    favoritos: [],
  };
[...]
```

```
  marcarFavorito(excursionId) {
    this.setState({favoritos: this.state.favoritos.concat(excursionId
)}});
  }
[...]
```

```
  <RenderExcursion
    excursion={this.state.excursiones[+excursionId]}
    favorita={this.state.favoritos.some(el => el === excu
rsionId)}

    onPress={() => this.marcarFavorito(excursionId)}
  />
[...]
```

```
const styles = StyleSheet.create({
[...]
```

```
  iconoContainer: {
    alignItems: 'center',
    marginBottom: 8,
  },
[...]
```

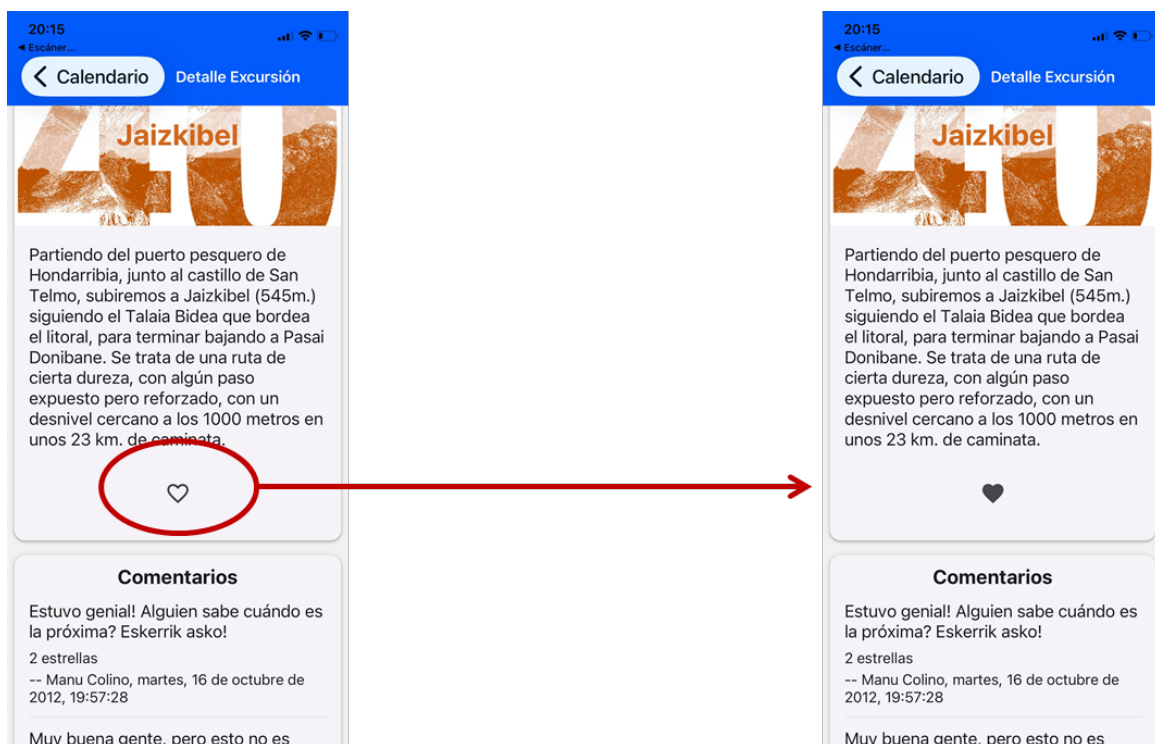
Cuestiones a comentar:

- IconButton (de React Native Paper)
 - Consultar la documentación para ver el efecto de cada uno de los parámetros que se han empleado, además de explorar otras funcionalidades.
- Del mismo modo que hacemos en cualquier aplicación *JavaScript* que se ejecute en el Navegador, en *React Native* también podemos mostrar información en la consola (no será visible en la aplicación), empleando el comando *console.log()*:

```
console.log('mensaje a mostrar')
```

- Nótese que la aplicación sólo es capaz de recordar si una excursión ha sido marcada como favorita dentro del componente *DetalleExcursion*. El array *favoritos[]* se reseteará cada vez que accedamos al componente *DetalleExcursion*, por lo que esta información se perderá en caso de volver atrás o salir de esta pantalla. En futuros ejercicios veremos la forma en que esta información puede almacenarse de forma permanente.
- Como siempre, es importante tener claro el código *JavaScript* que utilizamos en nuestra aplicación. En particular, en este ejercicio, el uso de *filter* y de *some*.

Una vez completados los dos primeros apartados de este capítulo, la vista de detalle de las excursiones quedaría como se muestra a continuación (la imagen superior se muestra recortada, para que se vea al completo el primer comentario):



3. Customizar el Drawer Menu

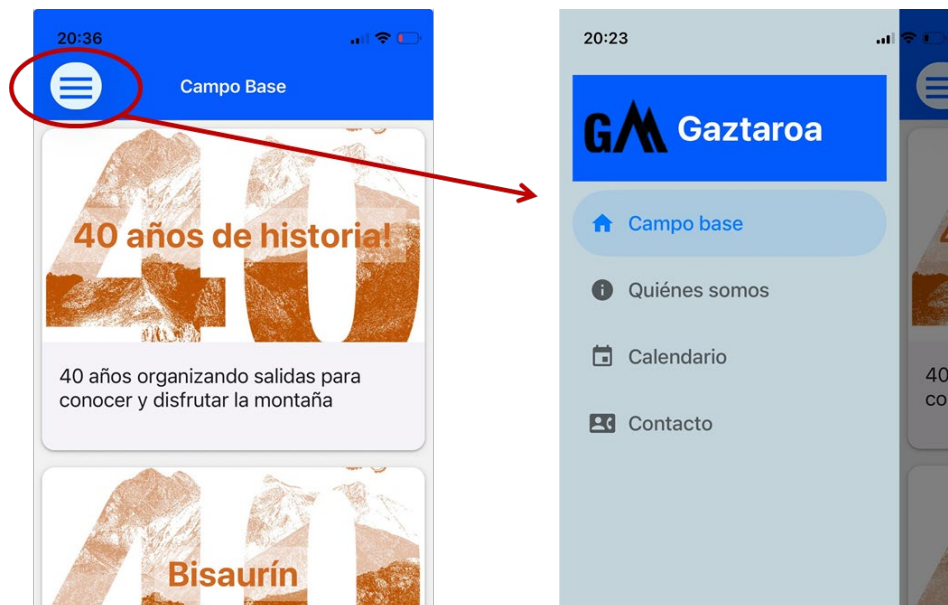
El objetivo de esta tercera sección es aprovechar lo que hemos aprendido hasta ahora, para customizar también el *Drawer Menu*, haciéndolo un poquito más atractivo.

Para ello, crearemos:

- Iconos a la izquierda de cada una de las opciones del menú.
- Una zona en la parte superior del menú, para mostrar el logotipo de Gaztaroa, además del nombre del club de montaña.

Adicionalmente, deseamos añadir un botón en la parte superior izquierda de cada una de las pantallas, cuya función será desplegar el *Drawer Menu*. Para ello, definiremos un pequeño componente de apoyo (*BotonMenu*) y una función auxiliar (*menuHeaderOptions*) que nos permitirá reutilizar la configuración común de la cabecera en las distintas pantallas. En dicha configuración haremos uso de la propiedad *headerLeft* y del método *DrawerActions.toggleDrawer()*. La excepción será la pantalla *DetalleExcursion*, en la que mantendremos el botón nativo para volver a la pantalla *Calendario*.

El resultado que estamos buscando es el siguiente:



A continuación, se muestran algunas de las modificaciones en el código fuente del fichero *CampobaseComponent.js* que nos permiten alcanzar dicho objetivo:

```
[...]
import { View, Platform, StyleSheet, Image, Text,
Pressable } from 'react-native';
import { createDrawerNavigator, DrawerContentScrollView, DrawerItemList
} from '@react-navigation/drawer';
import { MaterialCommunityIcons } from '@expo/vector-icons';
[...]

function BotonMenu(props) {
  return (
    <Pressable
      onPress={props.onPress}
      hitSlop={8}
    >
      <MaterialCommunityIcons
        name="menu"
        size={40}
        color={Platform.OS === 'ios' ? '#015afc' : 'white'}
      />
    </Pressable>
  );
}

function CustomDrawerContent(props) {
  return (
    <DrawerContentScrollView {...props}>
      <SafeAreaView style={styles.container} edges={['left', 'right',
'bottom']}>
        <View style={styles.drawerHeader}>
          <View style={styles.drawerHeaderImageContainer}>
            <Image
              source={require('./imagenes/logo.png')}
              style={styles.drawerImage}
            />
          </View>
          <View style={styles.drawerHeaderTextContainer}>
            <Text style={styles.drawerHeaderText}>Gaztaroa</Text>
          </View>
        </View>
        <DrawerItemList {...props} />
      </SafeAreaView>
    </DrawerContentScrollView>
  );
}
[...]
```

```
class Campobase extends Component {
[...]
```

```
  menuHeaderOptions = (title, navigation) => ({
    title,
    headerLeft: () => (
      <BotonMenu
        onPress={() => navigation.dispatch(DrawerActions.toggleDrawer())}
      />
    ),
  });

  HomeNavegador = () => {
    return (
      <Stack.Navigator
        initialRouteName="Home"
        screenOptions={{
          headerTintColor: 'fff',
          headerStyle: { backgroundColor: '#015afc' },
          headerTitleStyle: { color: 'fff' },
        }}
      >
        <Stack.Screen
          name="Home"
          component={Home}
          options={({ navigation }) =>
            this.menuHeaderOptions('Campo Base', navigation)
          }
        />
      </Stack.Navigator>
    );
  };
[...]
```

```
  DrawerNavegador = () => {
    return (
      <Drawer.Navigator
        initialRouteName="Campo base"
        drawerContent={(props) => <CustomDrawerContent {...props} />}
        screenOptions={{
          headerShown: false,
          drawerStyle: {
            backgroundColor: '#c2d3da',
          },
        }}
      >
        <Drawer.Screen
```



```
      name="Campo base"
      component={this.HomeNavegador}
      options={{
        drawerIcon: ({ color, size }) => (
          <MaterialCommunityIcons
            name="home"
            color={color}
            size={size}
          />
        ),
      }}
    />
```

[...]

```
const styles = StyleSheet.create({
  container: {
    flex: 1,
  },
  drawerHeader: {
    backgroundColor: '#015afc',
    height: 100,
    flexDirection: 'row',
    alignItems: 'center',
    marginBottom: 8,
  },
  drawerHeaderImageContainer: {
    flex: 1,
    justifyContent: 'center',
    alignItems: 'center',
  },
  drawerHeaderTextContainer: {
    flex: 2,
    justifyContent: 'center',
  },
  drawerHeaderText: {
    color: 'white',
    fontSize: 24,
    fontWeight: 'bold',
  },
  drawerImage: {
    width: 80,
    height: 60,
    resizeMode: 'contain',
  },
});

export default Campobase;
```

Importante, como siempre, analizar en detalle dicho código, con especial atención a las siguientes claves:

- SafeAreaView
- La opción *drawerContent* de *Drawer.Navigator* y las funciones asociadas.
- MaterialCommunityIcons
 - El código mostrado es el asociado a la opción “Campo base” del *Drawer Menu*. Para el resto de opciones se han elegido los siguientes iconos de MaterialCommunityIcons:
 - *'information'* (para la opción “Quiénes somos”)
 - *'calendar'* (para la opción “Calendario”)
 - *'card-account-phone'* (para la opción “Contacto”)
- En este punto hemos hecho un uso relativamente avanzado de los estilos en-línea que son la base de la customización de los componentes en React Native. Este es, por lo tanto, un buen momento para profundizar en el conocimiento de los mismos. Además, se ha customizado el *layout* de los diferentes elementos dentro de dicho menú, empleando la propiedad *Flex*. Se recomienda consultar las referencias que se muestran al final del documento para profundizar en ambos puntos.

Llegados a este punto, solo tenemos que comprobar que todo es correcto y hacer el *commit* correspondiente a este ejercicio.

4. Bibliografía

- **IconButton (React Native Paper)**
<https://callstack.github.io/react-native-paper/3.0//icon-button.html>
- **Expo Vector Icons**
<https://docs.expo.dev/guides/icons/>
- **Safe Areas in React Navigation**
<https://reactnavigation.org/docs/handling-safe-area/>
- **React Native Safe Area Context**
<https://github.com/AppAndFlow/react-native-safe-area-context>
- **Drawer Navigator (React Navigation)**
<https://reactnavigation.org/docs/drawer-navigator/>
- **DrawerActions / toggleDrawer (React Navigation)**
<https://reactnavigation.org/docs/drawer-actions/>
- **Style y StyleSheet en React Native**
<https://reactnative.dev/docs/style>
<https://reactnative.dev/docs/stylesheet>
- **Anchura y altura de componentes en React Native**
<https://reactnative.dev/docs/height-and-width>
- **React Native Layout con Flexbox**
<https://reactnative.dev/docs/flexbox>
<https://reactnative.dev/docs/layout-props>
<https://yogalayout.com/playground/>
- **Array.prototype.some() (JavaScript)**
https://developer.mozilla.org/es/docs/Web/JavaScript/Reference/Global_Objects/Array/some
- **Array.prototype.filter() (JavaScript)**
https://developer.mozilla.org/es/docs/Web/JavaScript/Reference/Global_Objects/Array/filter
- **Console Log / depuración en Expo y React Native**
<https://reactnative.dev/docs/debugging>
<https://docs.expo.dev/debugging/runtime-issues/>
- **Configuración de la barra de estado (Expo)**
<https://docs.expo.dev/versions/latest/sdk/status-bar/>